

Browser Execution Context for C2 Beacon Detection, and the Limits of Network-Flow Training Data

L. K. Wadduwage, Hansika Mahaadikara, and Amila Senarathne

Department of Computer Systems Engineering, Sri Lanka Institute of Information Technology, Malabe, Sri Lanka
it22208408@my.sliit.lk, hansika.m@sliit.lk, amila.s@sliit.lk

Abstract—A malicious browser extension that beacons to a command-and-control server produces traffic a network monitor cannot distinguish from ordinary browsing: periodic HTTPS requests to a reachable host, often a trusted platform. What a network monitor cannot see is the *execution context* — whether a human was interacting when the request fired, whether it came from a foreground tab, and whether an extension originated it. We present a detector that observes outbound requests from inside a managed Chromium session through the DevTools protocol, tags each with interaction and tab context collected by non-invasive instrumentation, and computes a 14-feature vector per host over a rolling window, four of whose features have no counterpart in any network capture. Scores from an anomaly model, threat-intelligence reputation and deterministic heuristics are fused with adaptive weights. Our evaluation is largely a negative result about training data. Supervised models trained on 681,911 windows derived from a public botnet capture reach an in-sample PR-AUC of 0.994 and ROC-AUC of 1.000, but under leave-one-scenario-out validation the same pipeline falls to a mean PR-AUC of 0.158 with mean precision 0.061. Worse, a cadence-trained model scores 0.947 on one beacon profile and 0.014 on the same beacon with 5% timing jitter, and 0.000 on all three live browser beacon profiles, where deterministic heuristics score 0.65–0.73. Across six hard-negative profiles of 200 samples each — SPA background sync, analytics beacons, video segments, extension update polling and push keep-alive — the deployed pipeline raises zero false beacons, while a candidate model with higher offline scores raises three.

Index Terms—command and control, beacon detection, browser security, execution context, anomaly detection, dataset shift, threat intelligence.

I. INTRODUCTION

Command-and-control beaconing is the step that turns a compromise into an operation. The implant contacts its server on a schedule, asks for instructions, and returns results. Detection has traditionally been a network problem, solved by looking for regularity: consistent inter-arrival times, consistent payload sizes, a destination contacted at a cadence no human would produce.

Browser extensions have moved that behaviour inside the browser, where the network view degrades badly. The extension's requests leave the host indexed by the browser's own TLS session, mixed with genuine browsing to the same destinations. A beacon to a chat platform, a code-hosting API or a document service is indistinguishable at the flow level from a user with that service open, and no amount of regularity analysis over encrypted flows recovers the attribution.

The observation this paper builds on is that the browser holds information the network cannot: whether the user was interacting with the machine at the instant a request fired, whether the originating tab was in the foreground, and whether the initiator was an extension rather than a page. A request that is perfectly periodic, fires while the user is idle, comes from a background context, and targets the same endpoint every time is a beacon regardless of what its destination's reputation says.

Our contributions are:

- A browser-execution-aware detector with four context features (idle time, user-active ratio, background-tab ratio, extension origin) that cannot be derived from network captures, collected without injecting hooks into page JavaScript (Section III).
- A quantified failure of transfer from network-flow training data to browser beacon detection, measured with leave-one-scenario-out validation and a timing-jitter sensitivity test (Section IV-C).
- A hard-negative battery of six benign periodic-traffic profiles that a plausible detector will flag, and evidence that the model with better offline metrics is the one that fails it (Section IV-E).

II. RELATED WORK

Flow-based beacon detection is well established. Regularity statistics over inter-arrival times, payload-size consistency and destination-entropy measures underpin both academic detectors and production tooling, and the CTU-13 capture set [1] remains the standard labelled corpus. Gu *et al.* [2] clustered communication and activity patterns to identify bots independently of protocol, and subsequent work has repeatedly shown that regularity alone is fragile once implants add jitter.

Two constraints motivate our design. First, encrypted transport removes payload inspection, leaving timing and volume; modern C2 frameworks add jitter precisely to defeat those. Second, the browser is a shared channel: attribution to an extension is impossible from outside. Work on extension-originated traffic has generally relied on static permission analysis rather than runtime attribution.

For the model itself, Isolation Forest [3] suits a setting where benign behaviour is abundant and labelled beacon samples are scarce and frameworks vary widely; a classifier fitted to one framework's cadence will miss another's by construction.

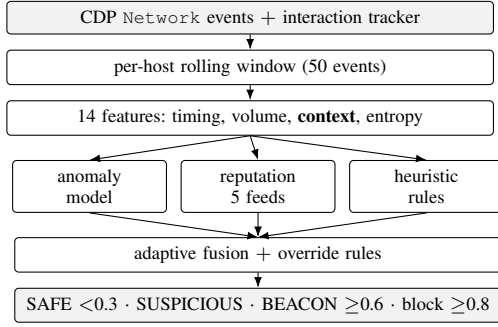


Fig. 1. Detection pipeline. Analysis runs every ten seconds per monitored host.

Threat-intelligence feeds provide a complementary, reactive signal.

III. SYSTEM DESIGN

A. Non-invasive collection

Requests are captured through the DevTools protocol’s network domain rather than through request interception. The distinction is a performance decision: protocol events are fire-and-forget, so the browser never waits for the analysis process, whereas a global route handler pauses every request — a hundred to three hundred per page load — until the handler responds. Interception is used only to block a confirmed beacon host, registered as a single per-host pattern.

User interaction is tracked by an init script listening for `click`, `keydown`, `scroll` and `touchstart`. Pointer movement is deliberately excluded: it fires from passive cursor presence, cursor-following animations and application focus changes, none of which evidence deliberate engagement, and including it inflates the user-active ratio and deflates measured idle time — degrading exactly the two most novel features. Because a persistent browser context restores previously open tabs that no init script will retroactively reach, the tracker is additionally evaluated directly into every already-loaded page at start-up.

Requests originating from service workers and other non-page contexts have no associated tab. Their initiator type is checked and they are attributed to the background with the user marked inactive, which computes the context features correctly without needing a page comparison.

B. Features

Table I lists the 14 features. F01–F08 and F13–F14 are the conventional timing, volume and entropy statistics, computable from a network capture. F09–F12 are the contribution: they exist only inside the browser and cannot be derived from any flow record, which is also why no public dataset can supply them.

C. Three signals, adaptive fusion

The anomaly model scores deviation from a benign browsing baseline. Reputation aggregates five threat-intelligence sources, taking the maximum across them, and is gated: it

TABLE I
PER-HOST FEATURE VECTOR

ID	Feature	Beacon signal
F01	inter-arrival mean	consistent interval
F02	inter-arrival CV	≈ 0 ; browsing 0.5–1.5
F03	Bowley skewness	timing asymmetry
F04	inter-arrival MAD	robust regularity
F05	requests per hour	volume normalisation
F06	payload mean	check-ins are small
F07	payload std. dev.	consistent sizes
F08	POST ratio	check-in/exfil alternation
F09	mean idle time	fires while user idle
F10	user-active ratio	≈ 0 ; browsing ≈ 1
F11	background-tab ratio	fires from background
F12	extension origin	direct attribution
F13	URL path entropy	same endpoint repeatedly
F14	burst count	exfiltration bursts

is queried only for hosts whose behavioural score already exceeds 0.2, because free API tiers permit as few as four requests per minute while active browsing touches dozens of hosts. Results are cached with a reputation-dependent lifetime. Deterministic heuristics score perfect timing regularity, idle firing, background origin, extension attribution and low path entropy, and provide coverage from the first run with no model and no API keys.

Fusion is $0.60a + 0.25r + 0.15h$ when all three are present, and redistributes weight when any is unavailable — $0.75a + 0.25h$ offline, $0.55h + 0.45r$ with no trained model. Two override rules prevent dangerous misses: a reputation score above 0.8 raises the result to at least the beacon threshold, as does strong agreement between a high anomaly score and a high heuristic score. A two-tier verdict rule allows only SUSPICIOUS from three or four observations and requires five before a host can be declared a beacon and blocked, unless reputation already confirms the destination.

IV. EVALUATION

A. Setup

Offline models were trained on windows derived from a public botnet capture set [1], restricted to HTTP flows. Two variants are compared: an *HTTP-flow model* using the full available feature set, and a *cadence-only model* restricted to timing and volume statistics. Because scenarios in that corpus differ in malware family, infrastructure and capture conditions, we evaluate with leave-one-scenario-out (LOSO) validation as well as a random split, and report only folds with enough positives to be meaningful. Browser behaviour is evaluated separately, on live beacon profiles and on hard-negative profiles of benign periodic traffic.

B. In-sample metrics are meaningless here

The production supervised model was fitted on 681,911 windows containing 492 positives, an imbalance near 1:1386 handled with a positive-class weight of 2770. It reports an in-sample PR-AUC of 0.994 and a ROC-AUC of 1.000. Its dominant feature is mean payload size at 0.477 importance —

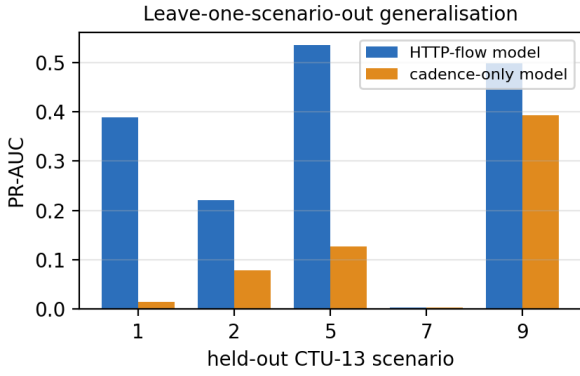


Fig. 2. Per-fold PR-AUC under leave-one-scenario-out validation. Performance is dominated by which capture is held out, not by the model.

TABLE II
RANDOM SPLIT VERSUS LEAVE-ONE-SCENARIO-OUT

Model / protocol	PR-AUC	Prec.	Rec.	F_1
HTTP-flow, random split	0.715	0.513	0.769	0.615
Cadence-only, random split	0.197	0.073	0.769	0.133
Cadence-only, LOSO mean	0.158	0.061	0.280	0.096

a capture artefact of one corpus, not a property of beaconing. We report these numbers only to dismiss them: they are the shape a memorised training set takes.

C. Transfer failure across scenarios

Table II and Fig. 2 give the result. A random split across all scenarios flatters both models, because windows from the same capture appear on both sides of the split. Held out by scenario, the cadence model averages a PR-AUC of 0.158 with a precision of 0.061 — 43 true positives against 515 false ones. Per-fold PR-AUC ranges from 0.003 to 0.393, so the dominant variable is which capture is withheld, not the model.

We also swept the positive-class weight from 1.0 to 3.0 under LOSO. Mean recall rises modestly from 0.280 to 0.317 while false positives climb from 515 to 841, and mean PR-AUC falls; the weight was left at 1.0. There is no setting at which this model becomes deployable on its own.

The jitter test is more damaging still. On four fixed beacon profiles the cadence model scores 0.947 and 0.984 on two regular ones, and 0.014 and 0.015 on otherwise identical beacons carrying 5% and 10% timing jitter. A model whose learned concept is cadence collapses the moment an implant adds the jitter that every modern framework offers as a configuration option.

D. Live browser beacons: the heuristic carries detection

Fig. 3 shows the deployed pipeline on three beacon profiles generated in the live browser. The learned component scores 0.0009, 0.000 and 0.000. The heuristic scores 0.71, 0.73 and 0.65, driven by the context features: the requests fire while the user is idle, from a background context, to one endpoint, at a fixed interval. The fused scores of 0.497, 0.511 and 0.455

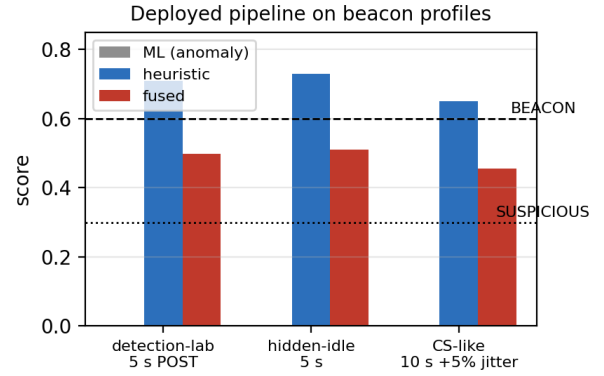


Fig. 3. Deployed pipeline on three browser beacon profiles. The learned component contributes essentially nothing; the fused verdict is produced by the deterministic rules.

TABLE III
BENIGN PERIODIC TRAFFIC, 200 SAMPLES PER PROFILE

Profile	Deployed		Candidate	
	FP	max	FP	max
Active human browsing	0	0.060	0	0.245
SPA background sync	0	0.358	1	0.593
Third-party analytics beacon	0	0.091	0	0.228
Video streaming segments	0	0.254	0	0.293
Extension filter-list update	0	0.436	1	0.630
Push keep-alive	0	0.511	1	0.597
Total	0	—	3	—

place all three in SUSPICIOUS — detected and surfaced, but below the automatic blocking threshold.

This is the honest position: on browser-originated beacons, the system detects through deterministic context rules, and the model trained on network flows contributes nothing. That is not a defect of the rules; it is a statement about what the available labelled data can teach.

E. Hard negatives, and why the better model is worse

Benign periodic traffic is abundant. Single-page applications poll, analytics libraries beacon on a timer, video players fetch segments at a fixed cadence, extensions update filter lists, and push services send keep-alives. Every one of these is regular, and several fire while the user is idle. Table III evaluates 200 samples of each.

The deployed pipeline raises zero false beacons across all 1,200 samples, with its highest score, 0.511, on push keep-alive — correctly suspicious, correctly below the beacon threshold. A candidate model that scored better offline, and that lifted one beacon profile from SUSPICIOUS to BEACON, raises three false beacons here. Its improvement on one true positive costs three false ones on the traffic a real user generates continuously. On that basis the candidate was not promoted, and the deployed configuration was left in place.

V. DISCUSSION

The through-line is that the available labelled data and the deployment environment are different domains, and the gap is not a matter of degree. Browser-context features cannot be back-filled into network captures, so the very features that carry detection are the ones no public corpus can supervise; and a model trained on the features that public corpora *do* carry learns a capture-specific cadence that jitter defeats. Random-split validation conceals this entirely, because same-capture windows land on both sides of the split; scenario-held-out validation is the minimum honest protocol for any corpus that is a union of captures.

Two practical consequences follow. First, a deterministic rule layer is not a placeholder to be removed once the model is trained — here it is the component that actually detects, and it should be weighted and defended accordingly. Second, offline improvement is not evidence for promotion. A hard-negative battery drawn from the benign traffic the system will meet continuously, rather than from a held-out slice of the training corpus, is what distinguishes a real improvement from a threshold shift.

VI. LIMITATIONS AND FUTURE WORK

The browser is launched without side-loaded extensions, so extension background service workers are separate targets not reachable through standard page protocol sessions; detection is demonstrated through a test beacon page that reproduces the same timing and context patterns, and loading real extensions is future work. The anomaly model requires a benign baseline collected from the deployed browser, and until enough is gathered the system runs on heuristics and reputation alone. Free threat-intelligence tiers impose rate limits that gating and caching mitigate but do not remove, and hosts behind shared content-delivery infrastructure cannot yield meaningful address reputation. Finally, the beacon profiles used here are generated rather than captured from live malware; validating against real extension-borne C2 samples is the most valuable next step.

VII. CONCLUSION

Beacon detection inside the browser is not the network problem moved indoors. The signals that separate a beacon from ordinary traffic are the ones the browser alone can see — who was interacting, which tab, which initiator — and the signals that public corpora can supervise are precisely the ones an implant perturbs with a jitter setting. We measured that gap rather than assuming it: an in-sample PR-AUC of 0.994 becomes 0.158 held out by scenario, a 0.947 detection becomes 0.014 under 5% jitter, and on live browser beacons the learned component contributes nothing while context heuristics score 0.65–0.73. The configuration we deploy is the one that raises no false beacons across 1,200 samples of the benign periodic traffic every user generates — not the one with the better offline number.

ACKNOWLEDGEMENT

The authors thank the members of research group R26-CS-003 for their collaboration on the shared WebSentinel platform, and the Stratosphere Laboratory and the maintainers of the public threat-intelligence feeds used in this work.

REFERENCES

- [1] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, “An empirical comparison of botnet detection methods,” *Computers & Security*, vol. 45, pp. 100–123, 2014.
- [2] G. Gu, R. Perdisci, J. Zhang, and W. Lee, “BotMiner: Clustering analysis of network traffic for protocol- and structure-independent botnet detection,” in *Proc. USENIX Security Symposium*, 2008, pp. 139–154.
- [3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.
- [4] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [5] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] Google, “Chrome DevTools Protocol.” [Online]. Available: <https://chromedevtools.github.io/devtools-protocol/>
- [7] Microsoft, “Playwright for Python.” [Online]. Available: <https://playwright.dev/python/>
- [8] AbuseIPDB, “IP address abuse reporting and checking.” [Online]. Available: <https://www.abuseipdb.com/>
- [9] AT&T Alien Labs, “Open Threat Exchange (OTX).” [Online]. Available: <https://otx.alienvault.com/>
- [10] abuse.ch, “URLhaus: Malware URL exchange.” [Online]. Available: <https://urlhaus.abuse.ch/>
- [11] abuse.ch, “Feodo Tracker botnet C2 blocklist.” [Online]. Available: <https://feodotracker.abuse.ch/>
- [12] B. E. Strom *et al.*, “MITRE ATT&CK: Design and philosophy,” MITRE Corporation, Tech. Rep., 2020.